

Lab 2: Signal handling (30.1.25)

Goal: In this lab you will learn to set up basic signals and signal handlers. Please make sure you get checked off on exercise 2.

Review Problems

Assume there are no errors (e.g. fork doesn't fail), but make no assumptions about how the processes are scheduled. Assume each call to printf flushes its output out just before it returns.

1. Consider the following code. What is the output?

```
int val = 10;
void handler(int sig) {
    val += 5;
}
int main() {
    pid_t pid;
    signal(SIGCHLD, handler);
    if ((pid = fork()) == 0) {
        val = val - 3;
        exit(0);
    }
    waitpid(pid, NULL, 0);
    printf("val = %d\n", val);
    exit(0);
}
```

Answer: val = 15 (not val = 10; explain why)

2. Consider the following code. What is the output?

```
pid_t pid;
int counter = 0;
void handler1(int sig)
{
    counter++;
    printf("counter = %d\n", counter);
    /* Flushes the printed string to stdout */
    fflush(stdout);
    kill(pid, SIGUSR1);
}

void handler2(int sig)
{
    counter += 3;
    printf("counter = %d\n", counter);
    exit(0);
}

main() {
    pid_t p; int status;

    signal(SIGUSR1, handler1);
    if ((pid = fork()) == 0) {
        signal(SIGUSR1, handler2);
        kill(getppid(), SIGUSR1);
        while(1) ;
    }
    if ((p = wait(&status)) > 0) {
        counter += 4;
        printf("counter = %d\n", counter);
    }
}
```

Answer: (unique sequence)

counter = 1 (parent's handler)

counter = 3 (child's handler)

counter = 5 (parent's main)

3. For the code segment below, give the **largest** value that could be printed to stdout.

```
/* Version A */
int i = 0;
void handler(int s) {
    if (!i) {
        kill(getpid(), SIGINT);
    }
    i++;
}

int main() {
    signal(SIGINT, handler);
    kill(getpid(), SIGINT);
    printf("%d\n", i);
    return 0;
}
```

Answer: 2 (other possible outputs 0 and 1)

4. For the code segment below, give the **largest** value that could be printed to stdout.

```
/* Version B */
int i = 0;
void handler(int s) {
    if (!i) {
        kill(getpid(), SIGINT);
        kill(getpid(), SIGINT);
    }
    i++;
}

int main()
{
    signal(SIGINT, handler);
    kill(getpid(), SIGINT);
    printf("%d\n", i);
    return 0;
}
```

Answer: 2 (not 4; explain why?)

Exercise 1 – Orphan and Zombie Process

(a) Enter and compile the following program into an executable called zombie.

```
/* zombie.c */

#include <sys/types.h>
#include <unistd.h>

int main()
{
    if (fork() > 0) {      /* Line A */
        while (1);
    }
    return 0;
}
```

In your shell, execute zombie in the background (append your command with &):

```
./zombie &
```

Now run ps to examine your processes:

```
ps -f
```

You should see a process labeled as defunct. What does this mean? Explain how the program above would lead to a defunct process (also known as zombie process)

(b) To see why such a process is called zombie process, issue the command kill to terminate the process, by passing in the process ID of the defunct process. For example, if the defunct process has process ID 1234, issue command:

```
kill 1234
```

Run ps -f again to see if the defunct process is still there. To return to normal, kill the parent of the defunct process.

(c) Make a copy of zombie.c and name it orphan.c. In orphan.c, change the line labeled Line A to:

```
if (fork() == 0) {
```

Compile and name your executable orphan. In your shell, execute orphan in the background, and examine your processes just like Part (a). What is the name of the parent process of the process orphan? Explain why the parent process is different from the parent process of the non-defunct process zombie in Part (a).

Exercise 2 (to be checked off)

Applications can explicitly block and unblock selected signals using the `sigprocmask` function. Read the following man pages:

- https://www.gnu.org/software/libc/manual/html_node/Signal-Sets.html
- https://www.gnu.org/software/libc/manual/html_node/Process-Signal-Mask.html

Write a program `sigblock.c` that will have two loops, each of which simply prints the integers from 1 to 5 at 1 second intervals. During the first loop, `SIGINT` and `SIGQUIT` are blocked. During the second loop only `SIGINT` is blocked. While the program is running, you can try `^C` and/or `^\` to see if they are blocked as expected.